RL-Course 2025/26: Final Project Report

DRAI: Jannik Rombach, Adriano Polzer

February 26, 2026

1 Introduction

Reinforcement learning in adversarial settings poses unique challenges where an agent must not only learn to act effectively, but to do so against an opponent that actively counters its behavior. In this work, we train agents for a 1v1 hockey environment inspired by air hockey, where two players compete on opposite sides of a rink to score goals against each other. The game requires a combination of offensive and defensive skills across a fully continuous action space, with scoring goals as the ultimate objective.

Generalizing across different opponent play styles proves to be the central difficulty. We address this through reward shaping to accelerate early learning, exploration design, and most importantly a self-play curriculum that progressively challenges the agent with a diverse and evolving pool of opponents. We find that the diversity of this pool, rather than any single algorithmic choice, is the primary driver of agent robustness.

We implement and compare two established off-policy algorithms, Twin Delayed Deep Deterministic Policy Gradient (TD3) and Soft Actor-Critic (SAC), and analyze their behavior under different training conditions. The algorithms are first validated on standard continuous control benchmarks before being applied to the hockey environment. Methodology is covered in Section 2, experimental results in Section 3, and conclusions in Section 4. The full code and configuration files are available at `https://github.com/JannikRom/drai-rl`.

2 Method

The hockey environment's fully continuous action space makes discrete-action methods such as Deep Q-Network (DQN) [8] inapplicable, and on-policy algorithms such as Proximal Policy Optimization (PPO) [10] less attractive due to poor sample efficiency. We therefore adopt two off-policy, continuous-control algorithms that share a common backbone yet represent complementary design philosophies: TD3 [2] optimizes a deterministic policy with explicit bias-correction mechanisms, while SAC [3] optimizes a stochastic policy through entropy maximization. Both maintain a replay buffer $\mathcal{D}$ of transitions $(s_t, a_t, r_t, s_{t+1}, d_t)$, train an actor π and one or more critics Q_θ on minibatches from $\mathcal{D}$, and stabilize learning via slowly updated target networks through Polyak averaging,

$$\theta' \leftarrow \tau\,\theta + (1-\tau)\,\theta', \qquad \phi' \leftarrow \tau\,\phi + (1-\tau)\,\phi', \tag{1}$$

where $\tau \ll 1$ ensures the target parameters θ' and ϕ' track their online counterparts θ and ϕ slowly, providing a stable and consistent learning target throughout training. The individual algorithms are described in Sections 2.1 and 2.2, further improvements to both are presented in Section 2.3.

2.1 Twin Delayed Deep Deterministic Policy Gradient

TD3 [2] extends Deep Deterministic Policy Gradient (DDPG) [7] by adressing two fundamental failure modes of determenistic actor-critic methods: overestimation bias in the critic and instability in polciy updates. The actor encodes a *deterministic* policy $a = \pi_\phi(s)$ and is trained to maximize expected discounted return via the determenistic policy gradient theorem [12]:

$$\nabla_\phi J(\phi) \approx \mathbb{E}_{s\sim\mathcal{D}}\Big[\nabla_a Q_\theta(s,a)\big|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)\Big]\,. \tag{2}$$

In actor-critic methods, the policy is updated greedily with respect to the critic, meaning any overestimation in the value function is directly exploited by the actor, leading to suboptimal or diverging behaviour [2].
TD3 adresses this with three interacting mechanisms.

Clipped double Q-learning
Two independent critics Q_{θ_1} and Q_{θ_2} are trained in parallel but share no parameters. When computing Bellman targets, the minimum of the two target critics is used:

$$y = r + \gamma(1-d)\min_{i=1,2} Q_{\theta'_i}(s', \tilde{a}) \tag{3}$$

where d indicates episode termination and θ_i are the target network parameters updated via (1). Taking the minimum introduces a conservative bias that counteracts the systematic overastimation inherent in maximizing over noisy function approximators. Both critics are updated independently by minimizing the mean squarred Bellman error $(y - Q_{\theta_i}(s,a))^2$ over minibatches from D.
Since gradients flow independently through each critic, the two networks diverge slightly, preventing the correlated overestimation that arises when a single critic is used.

Target policy smoothing
To prevent the actor from exploiting narrow, misleading peaks in the value function, the target action is perturbed with clipped Gaussian noise before evaluating the Bellman target:

$$\tilde{a} = \mathrm{clip}\big(\pi_{\phi'}(s') + \epsilon,\ a_{\min},\ a_{\max}\big)\,, \quad \epsilon \sim \mathrm{clip}(\mathcal{N}(0,\sigma),\ -c,\ c)\,. \tag{4}$$

This encourages the critics to learn a smoother value landscape, making individual erroneous peaks less exploitable. The noise standard deviation σ and clip range c are hyperparameters that control the effective radius of this smoothing.

Delayed policy updates
Even with conservative targets and smoothed backups, updating the actor at every critic step introduces high variance into the policy gradient: the critic has not yet fully integrated the latest target information before its gradient is used to move the policy. TD3 therefore updates the actor every K critic steps, typically $K = 2$. This delay ensures the critics are in a more stable and accurate state when the actor gradient is computed, substantially reducing variance of policy updates and preventing the actor from over-adapting to transient critic errors [2]. Target networks for both actor and critics are updated via the shared Polyak rule (1), ensuring the learning targets change slowly and the training signal remains consistent across consecutive gradient steps.

Togehter, these three mechanisms directy counteract the core failure modes of deteremenistic actor-critic methods: clipped double Q-learning reduces overestimation bias, target policy smoothing regularizes the value function against sharp local peaks, and delayed updates reduce gradient variance during policy optimization.

2.2 Soft Actor-Critic

SAC is an off-policy actor-critic algorithm designed for optimizing *stochastic* policies in continuous action spaces. Our implementation follows the framework introduced by [3], which augments the standard maximum reward objective with an entropy maximization term [13]. This approach encourages wider exploration and improves robustness by seeking a policy that maximizes the trade-off between expected return and entropy:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(\mathbf{s}_t,\mathbf{a}_t)\sim\rho_\pi}[r(\mathbf{s}_t,\mathbf{a}_t) + \alpha\,\mathcal{H}(\pi(\cdot|\mathbf{s}_t))] \tag{5}$$

where ρ_π denotes the state-action distribution induced by policy π and $\mathcal{H}$ represents the entropy of the policy at state $\mathbf{s}_t$.

The SAC algorithm consists of an actor, critic, and an entropy regularization term governed by a temperature parameter α. The actor is parameterized by a neural network that defines a Gaussian distribution over the continuous action space by estimating the mean $\mu_\theta(s)$ and standard deviation $\sigma_\theta(s)$ in its output layer.

To ensure differentiability of the sampling process with respect to the policy parameters, the reparameterization trick is applied [3, 6]. Instead of sampling actions directly from the Gaussian distribution, actions are expressed as $a = \mu_\theta(s) + \sigma_\theta(s) \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, I)$ is a standard normal random variable. The sampled actions are passed through a $\tanh$ function and scaled to match the action bounds of the environment. To correctly account for this squashing transformation in the entropy term, the log-probability is adjusted using the determinant of the Jacobian of the $\tanh$ transformation. The actor is trained on the maximum entropy objective [3]:

$$J(\pi_\phi) = \mathbb{E}_{(\mathbf{s}_t\sim\mathcal{D})}\Big[\mathbb{E}_{(\mathbf{a}_t)\sim\pi_\phi}[\alpha \log \pi_\phi(a_t|s_t) - Q_\theta(s_t,a_t)]\Big] \tag{6}$$

where for each state sampled from the replay buffer $\mathcal{D}$, an action is sampled from the actor and the critic estimates the value for each state-action pair.

As presented in the follow-up work [4], we employ two Q-networks for the critic instead of a single Q-network and a value network as originally proposed. The soft value function can be implicitly derived from the soft Q-function by combining the Q-values with the policy entropy term.

During training, two independent Q-networks estimate the value of a state-action pair. To mitigate overestimation bias, soft policy evaluation is performed using the minimum of the two Q-values, which provides a more conservative value estimate [2]. The critic is trained using a mean squared Bellman error loss:

$$J_Q(\theta_i) = \mathbb{E}_{\mathcal{D}}\Big[(Q_{\theta_i}(s_t,a_t) - y_t)^2\Big] \tag{7}$$

with target value

$$y_t = r_t + \gamma\left(\min_{j=1,2} Q_{\bar{\theta}_j}(s_{t+1},a_{t+1}) - \alpha \log \pi_\phi(a_{t+1}|s_{t+1})\right). \tag{8}$$

The target networks $\bar{\theta}_j$ are updated using soft updates with factor τ, which improves training stability by preventing rapid divergence between online and target critics.

Furthermore, automatic parameter tuning is implemented as introduced in the follow-up paper [4]. The temperature parameter α is optimized to match a target policy entropy, allowing the agent to automatically balance exploration and exploitation. The temperature is updated by minimizing

$$J(\alpha) = \mathbb{E}_{a_t\sim\pi_\phi}\left[-\alpha\left(\log \pi_\phi(a_t|s_t) + \bar{\mathcal{H}}\right)\right], \tag{9}$$

where $\bar{\mathcal{H}}$ denotes the target entropy, typically set to $-\dim(\mathcal{A})$ as recommended in [4]. The parameter α is optimized using a gradient-based method.

2.3 Improvements

We implemented a framework based on YAML Ain't Markup Language (YAML) configuration files to manage and persist training runs, enabling reproducible experiments and systematic tracking of hyperparameter changes. All modifications described below are applied on top of the TD3 and SAC base implementations.

Replay Buffer

Since both implemented algorithms are off-policy, training relies on data stored in a replay buffer. We implemented standard Experience Replay (ER), which improves data efficiency by reusing past interactions and reducing correlations between subsequent samples. Transitions are stored as tuples (s_t, a_t, r_t, s_{t+1}) in a finite buffer and sampled uniformly during training. However, uniform sampling treats all transitions equally, ignoring that transitions with large Temporal Difference (TD) errors may provide stronger learning signal. To address this, we additionally implement Prioritized Experience Replay (PER) [9].

PER assigns higher sampling probabilities to transitions with larger TD errors. Each transition i receives priority $p_i = |\delta_i| + \varepsilon$, and is sampled according to $P(i) = p_i^\alpha / \sum_k p_k^\alpha$, where α controls the degree of prioritization. The resulting bias is corrected via importance-sampling weights $w_i = (N^{-1} \cdot P(i)^{-1})^\beta$, with β annealed linearly to 1 over training. Efficient sampling and updates are implemented using a binary SumTree in $\mathcal{O}(\log N)$ time.

Pink Noise

Exploration in off-policy algorithms is commonly achieved by injecting noise into the actor's actions. Standard white noise $\mathcal{N}(0, \sigma)$ is temporally uncorrelated, leading to jittery exploration with no coherence across consecutive actions. Eberhard et al. proposed colored noise as a principled alternative, parameterized by a spectral exponent β [1]. It is generated by scaling the Fourier components of a white noise signal by $f^{-\beta/2}$, suppressing high frequencies to produce temporally correlated action perturbations. Pink noise ($\beta = 1$) was shown to best balance exploration and exploitation across continuous control tasks. We apply it as the exploration noise for TD3, replacing the standard Gaussian perturbation. SAC learns a stochastic policy with explicit entropy regularization and therefore does not require external exploration noise.

Reward Shaping

To enhance learning in the hockey environment, we extend the default reward formulation with additional shaping terms. The environment provides sparse rewards for winning and losing, with puck contact and velocity signals disabled by default. The total reward is given by $r = \sum_i w_i \cdot r_i$, where each r_i denotes a reward component and $w_i \in \mathbb{R}$ its corresponding weight.
In early training, the agent often required many timesteps before initiating puck contact. To accelerate this, we introduce a time-penalty r_{passive}, applied when the puck remains stationary on the agent's side beyond the first 10% of episode steps,

$$r_{\text{passive}}(t) = \begin{cases} -c & \text{if } x_{\text{puck}} \leq 0,\ \|v_{\text{puck}}\| < 0.1,\ \text{and } t > 0.1 \cdot T_{\max}, \\ 0 & \text{otherwise,} \end{cases} \tag{10}$$

where $c > 0$ is a configurable penalty coefficient.

To encourage defensive positioning, we additionally reward the agent for staying close to its own goal when the puck is on the opponent's side,

$$r_{\text{defense}} = \begin{cases} \dfrac{1}{1 + \|p_{\text{agent}} - g\|_2} & \text{if } x_{\text{puck}} > 0, \\ 0 & \text{otherwise,} \end{cases} \tag{11}$$

where g denotes the agent's goal center. All shaping components are individually configurable via the YAML configuration file.

Self-Play

To enable the agent to continuously improve against increasingly strong opposition, we implement a self-play training framework inspired by AlphaZero [11]. Rather than training against a fixed opponent, the agent faces a dynamically growing **opponent pool** that diversifies the training distribution and prevents overfitting to any single playing style.

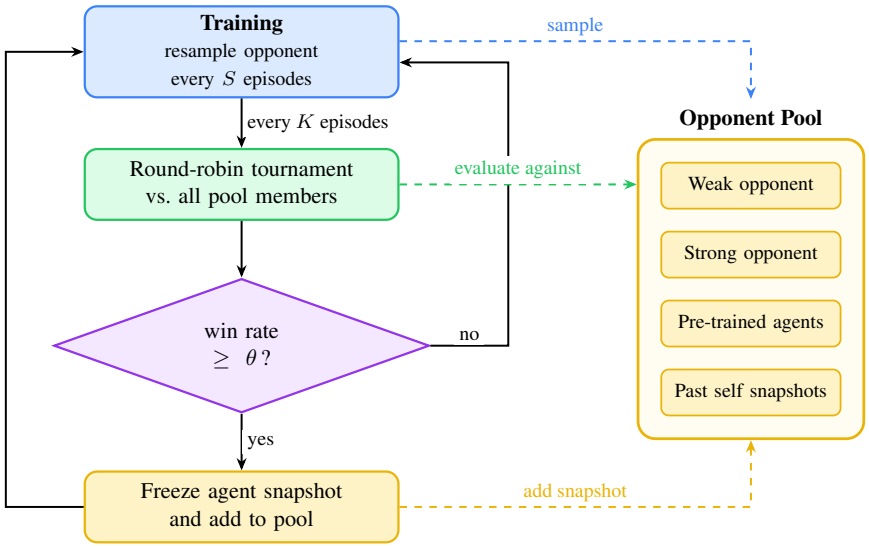

Figure 1: Self-play pool: Round-robin eval every K episodes vs. pool. Add snapshot if win rate $\geq \theta$.

The pool is initialized with a weak and a strong basic opponent and retains them permanently throughout training, serving as a minimum performance bar that the agent must reliably beat. Beyond these fixed baselines, the pool can also contain pre-trained agents of a different algorithm type, providing diverse and competent opposition.

During training, the agent's opponent is resampled every S episodes. Every K episodes, competence is assessed via a round-robin tournament where we evaluate against all pool members. The empirical win rate is

$$\hat{\rho} = \frac{W}{|\mathcal{P}| \cdot n_{\text{eval}}}, \tag{12}$$

where W is the number of wins, $|\mathcal{P}|$ the pool size, and n_{eval} the number of evaluation episodes per opponent. If $\hat{\rho} \geq \theta$, the agent's weights are frozen and we add snapshot to the pool, ensuring only genuinely stronger versions enter. To sample from the pool, a recency bias assigns higher probability to more recent snapshots, keeping the training challenge relevant while retaining older opponents for diversity. This creates a natural curriculum of continuously escalating opposition.

3 Evaluation

We evaluate both algorithms across two settings: standard continuous control benchmarks to validate the base implementations, and the hockey environment to assess the full pipeline including reward shaping, exploration noise, and self-play.

3.1 Simple Environments

We evaluate both SAC and TD3 in three standard benchmark environments: Pendulum, Lunar Lander, and Half Cheetah. For each algorithm, we compare training with standard ER and with PER in order to assess the effect of prioritized sampling on learning dynamics and final performance.
All algorithmic variants converge in Pendulum (200k steps) and LunarLander (800k steps), reaching comparable final returns as shown in Table 1. In these environments, PER does not lead to substantial performance differences.
In contrast, clearer distinctions emerge in HalfCheetah (800k steps). Both SAC and TD3 benefit substantially from prioritized replay, clearly outperforming their respective ER variants. Among all configurations, SAC with PER achieves the highest final results in this environment (Table 1). The results further indicate that SAC generally converges faster than TD3 across environments, as reflected in steeper initial learning curves (see Appendix A.1).

Table 1: Final Reward (Avg100 $\pm$ std)

	SAC + ER	**SAC + PER**	**TD3 + ER**	**TD3 + PER**
Pendulum	-145.96 $\pm$ 25.60	-144.72 $\pm$ 24.65	-147.11 $\pm$ 28.30	-150.81 $\pm$ 29.23
LunarLander	278.91 $\pm$ 15.18	261.56 $\pm$ 28.98	278.96 $\pm$ 12.55	273.12 $\pm$ 22.25
HalfCheetah	7580.42 $\pm$ 182.38	8908.11 $\pm$ 159.37	1726.27 $\pm$ 23.76	6421.28 $\pm$ 240.71

3.2 Hockey Environment

After validating the algorithms on standard benchmarks, we evaluate them in the hockey environment. Each variant is trained for 2.5×10^6 timesteps against a mixture of the weak and strong basic opponent without self-play. Evaluation rewards are computed exclusively against the strong opponent using the sparse reward (-10 loss, 0 draw, $+10$ win). Figure 2 shows learning curves for a representative subset of tested configurations.

SAC converges reliably and quickly due to its automatic entropy regularization and effective off-policy data reuse. Adding reward shaping accelerates early learning, as penalizing passivity and rewarding puck contact guide the agent to engage with the puck sooner. However, aggressively weighted shaping rewards may partially overshadow the dominant win signal, causing the agent to prioritize intermediate behaviors over scoring. Based on these observations, we adopt mild reward shaping to preserve the primacy of the win signal.

TD3 with standard white noise converges slower than SAC, as the deterministic policy relies entirely on externally injected noise for exploration. Replacing white noise with pink noise yields a clear improvement: the temporal correlation encourages the agent to commit to exploratory directions over consecutive timesteps, producing more purposeful puck-approach behavior and faster convergence [1]. Contrary to expectations, TD3 with PER converges slower than TD3 Basic: priorities change rapidly early in training, and the sampling distribution may become dominated by a small set of transitions, an instability we assume outweighs the benefit of focused sampling here. All variants ultimately converge and reliably beat the strong opponent.

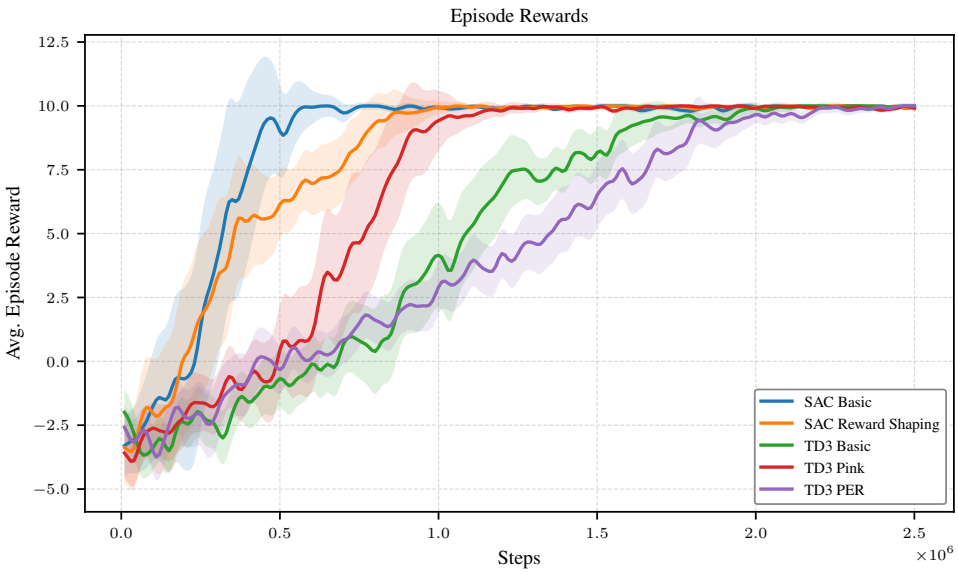

Figure 2: Avg. episode reward (mean $\pm$ eval variability).

Additional diagnostics including actor and critic losses and the SAC entropy temperature α are provided in Appendix B.1. Based on these experiments, both SAC and TD3 were configured with low-weight reward shaping, standard experience replay, and for TD3 pink noise exploration. Deeper networks with two to three hidden layers of moderate width outperformed shallower alternatives, while going too deep or too wide introduced instability. These configurations serve as the baseline for self-play fine-tuning, with full hyperparameters in Tables 6 and 7.

Self-Play and Tournament Evaluation

Both SAC and TD3 were subsequently fine-tuned with the self-play framework described in Section 2.3, where the growing opponent pool continuously challenges the agent with stronger versions of itself.
To evaluate agents throughout the development process, we implemented a round-robin tournament in which every participant plays n episodes as both the left and right player against every other participant. Rankings are determined via TrueSkill [5], a Bayesian rating system updating a skill estimate μ_i and uncertainty σ_i after every matchup, providing more robust rankings than simple win-rate averaging. Training progress was tracked via TensorBoard, logging standard metrics alongside self-play specific quantities such as win rate against the pool and pool size over time, which proved valuable for monitoring curriculum progression.

Inspecting individual games of top-ranked agents revealed a recurring pattern: several agents had learned to exploit a specific shooting angle, deflecting the puck off the wall to score. While effective against many opponents, these agents lost unexpectedly to lower-ranked agents with a different play style, a clear sign of overfitting to a narrow strategy. To address this, we fine-tuned our best existing agents against a pre-trained opponent pool containing checkpoints of competent SAC and TD3 agents trained under different seeds and configurations. A slightly reduced learning rate preserved previously learned behavior while adapting to the more diverse opposition.

The final tournament results are provided in Table 8 in the appendix. Agents trained with the pre-trained opponent pool consistently ranked highest, confirming that stylistic diversity in the training curriculum is key to producing strong and generalizable agents.

4 Discussion

Both SAC and TD3 proved to be well-suited algorithms for the hockey environment. SAC converged faster and required less manual tuning due to its automatic entropy regularization, while TD3 with pink noise exploration provided a competitive alternative. Hyperparameter tuning required considerable effort, but the most significant performance gains ultimately came from longer training runs and exposure to diverse opponents rather than from any single algorithmic choice.
The modular YAML-based framework proved valuable throughout the project, allowing rapid iteration over configurations without code changes. Combined with TensorBoard logging, we maintained a clear picture of training progression at all times. The round-robin tournament with TrueSkill ranking provided a reliable and interpretable comparison across all agent variants at every stage of development.

The most impactful finding was that training against a diverse pre-trained opponent pool led to substantially stronger and more generalizable agents. Agents trained only against the basic opponents or against pure self-play snapshots tended to overfit quickly, converging to narrow strategies that exploited specific weaknesses of familiar opponents. This overfitting became apparent only through the tournament, where top-ranked agents unexpectedly lost to stylistically different lower-ranked opponents. The final SAC and TD3 agents trained against the pre-trained pool win consistently against both the weak and strong basic opponent, as reflected in the win rates reported in Table 2. In hindsight, we would have invested more time earlier in building a larger and more varied pool, including agents trained under more diverse configurations and potentially additional algorithm types, to capture an even broader range of play styles.

Table 2: Win rates of the final agents against the basic opponents (1000 episodes each).

Agent	**Opponent**	**Win Rate**	**W-L-D**
SAC (pre-trained pool)	Basic Weak Opponent	99.2%	992-3-5
SAC (pre-trained pool)	Basic Strong Opponent	99.0%	990-9-1
TD3 (pre-trained pool)	Basic Weak Opponent	98.6%	986-5-9
TD3 (pre-trained pool)	Basic Strong Opponent	98.8%	988-10-2

Our results strongly suggest that exposure to diverse opponents is the primary driver of generalization. For the live tournament against the other course participants, fine-tuning against recorded opponent trajectories between matches could therefore be a particularly effective strategy to gain ranking, as it would allow the agent to adapt directly to the playing styles encountered in the competition.

Overall, the project offered valuable practical insight into the challenges of reinforcement learning that goes well beyond theoretical study alone. The hands-on nature of iteratively training, analyzing games, and refining strategies made the underlying complexity of the problem genuinely concrete. The infrastructure built around the training pipeline provided a solid foundation that made this iterative process efficient, reproducible, and enjoyable.

References

[1] O. Eberhard, J. Hollenstein, C. Pinneri, and G. Martius. Pink noise is all you need: Colored noise exploration in deep reinforcement learning. In *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*, May 2023.

[2] S. Fujimoto, H. van Hoof, and D. Meger. Addressing function approximation error in actor-critic methods. In J. Dy and A. Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 1587–1596, Stockholmsmässan, Stockholm Sweden, 10–15 Jul 2018. PMLR.

[3] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *International Conference on Machine Learning (ICML)*, 2018.

[4] T. Haarnoja, A. Zhou, K. Hartikainen, G. Tucker, S. Ha, J. Tan, V. Kumar, H. Zhu, A. Gupta, P. Abbeel, and S. Levine. Soft actor-critic algorithms and applications. *CoRR*, 2018.

[5] R. Herbrich, T. Minka, and T. Graepel. Trueskill™: A bayesian skill rating system. In *Advances in Neural Information Processing Systems 19*, pages 569–576. MIT Press, 2006.

[6] D. P. Kingma and M. Welling. Auto-encoding variational bayes. *International Conference on Learning Representations (ICLR)*, 2014.

[7] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[8] V. Mnih, K. Kavukcuoglu, D. Silver, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[9] T. Schaul, J. Quan, I. Antonoglou, and D. Silver. Prioritized experience replay. *arXiv: Learning*, 2015.

[10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.

[11] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.

[12] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller. Deterministic policy gradient algorithms. In *Proceedings of the 31st International Conference on Machine Learning (ICML-14)*, pages 387–395. JMLR Workshop and Conference Proceedings, 2014.

[13] B. D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

A Simple Environment Diagnostics

This appendix presents additional training diagnostics and hyperparameter settings for each evaluated simple environment. Training curves illustrate reward progression, loss, and the SAC specific entropy temperature.

A.1 Training Curves

Pendulum

Figure 3 shows additional training diagnostics for the Pendulum environment. All algorithms converge with respect to reward, actor loss, and critic loss, while SAC typically converges slightly faster. The SAC entropy temperature α behaves similarly for PER and ER variants and decreases over time, reflecting increasing policy confidence.

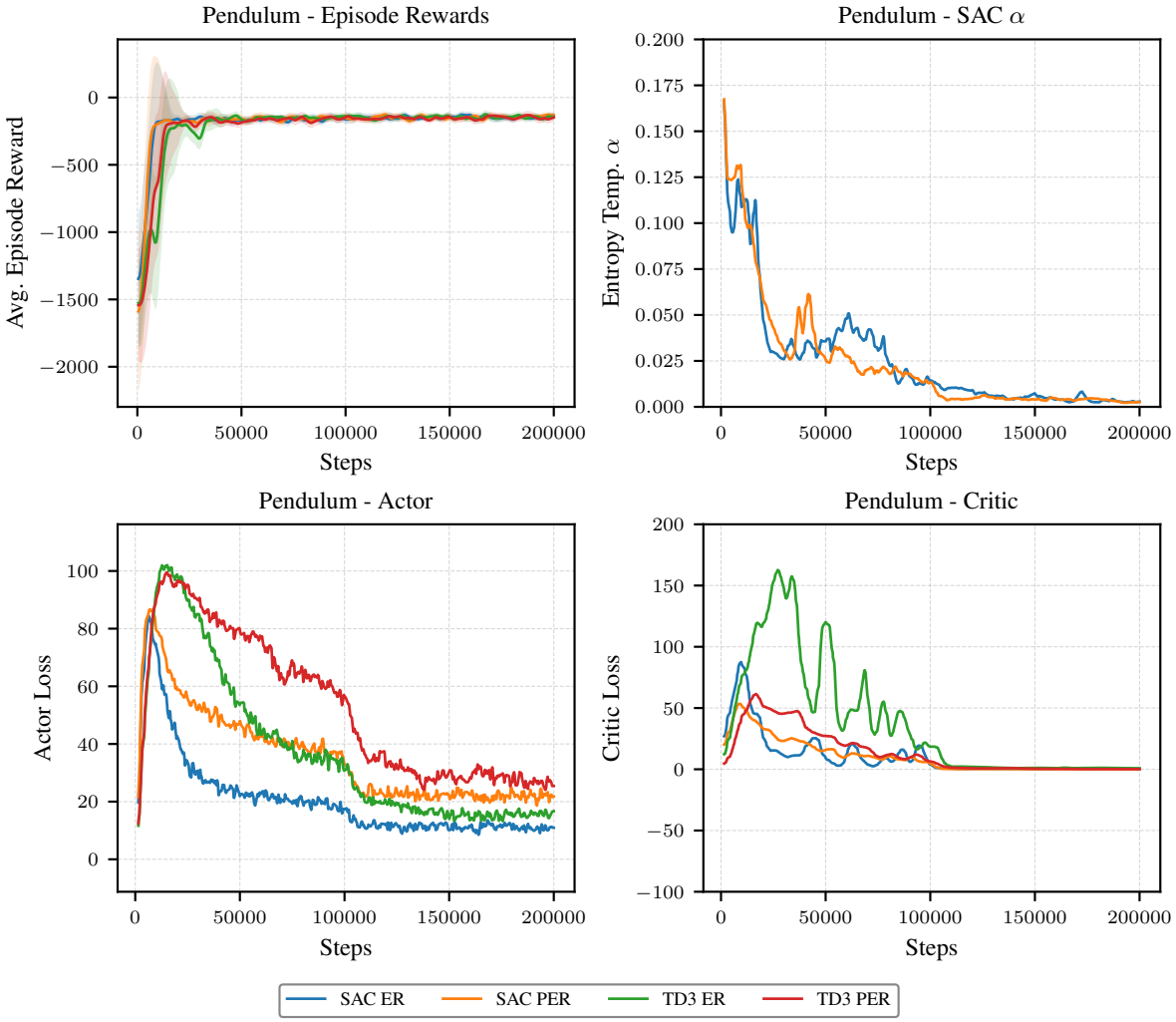

Figure 3: Training diagnostics for Pendulum environment. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

Lunar Lander

Training diagnostics for Lunar Lander are shown in Figure 4. Both algorithms exhibit similar reward convergence behavior. The SAC actor loss decreases faster than TD3, while TD3 with PER remains high for a longer period during training. The entropy temperature for PER remains noticeably higher compared to ER, indicating increased exploration pressure throughout training.

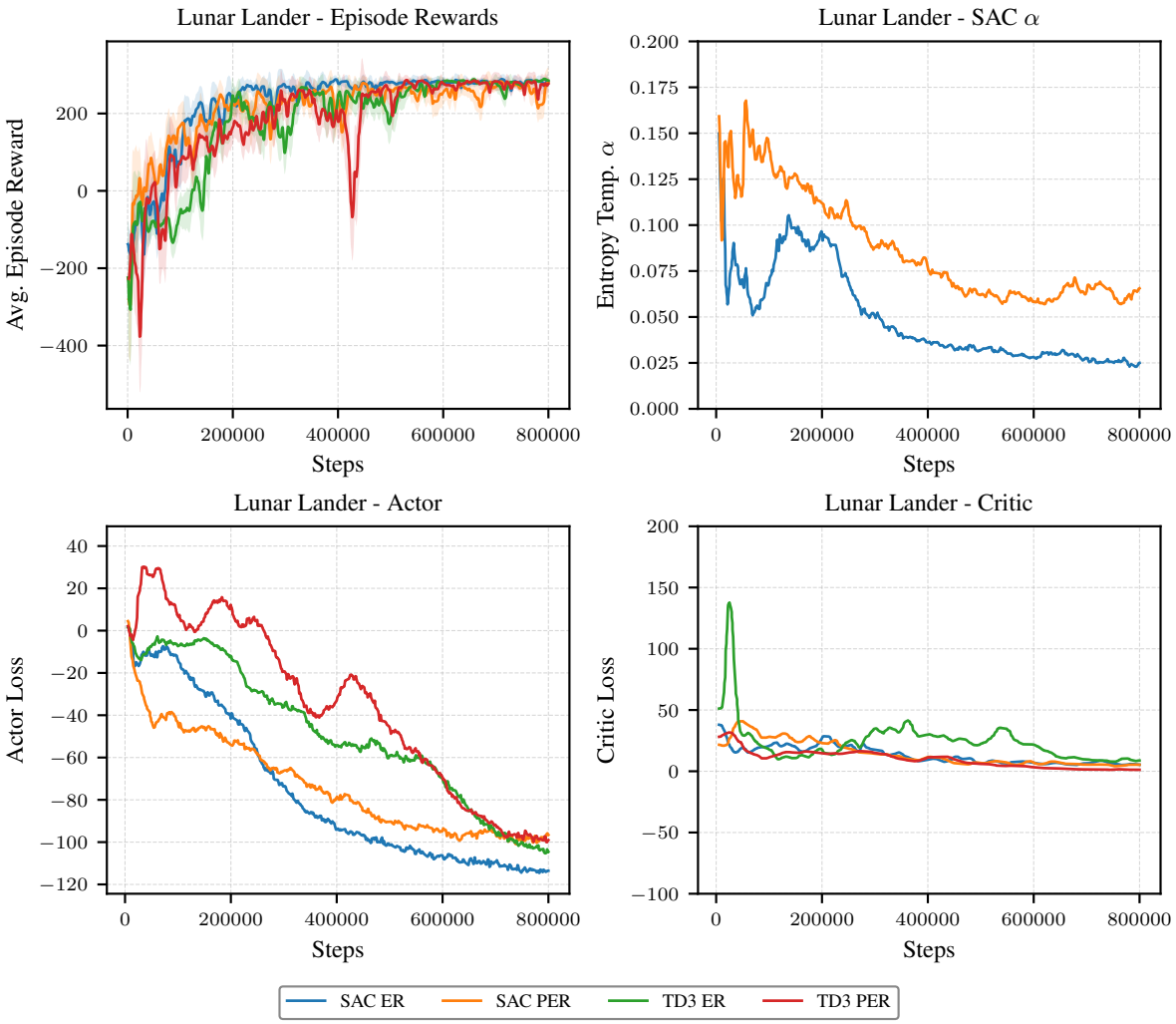

Figure 4: Training diagnostics for Lunar Lander environment. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

Half Cheetah

Training diagnostics for Half Cheetah are shown in Figure 5. All methods converge more slowly compared to the other environments. The highest rewards are achieved by SAC with PER, while TD3 achieves comparable performance to SAC with ER. However, TD3 with ER remains significantly lower in terms of final reward.
Actor losses behave similarly across most configurations, except for TD3 with ER, which exhibits a noticeably higher loss. For the critic loss, the TD3 ER variant again shows lower values compared to the other methods, while SAC with ER is slightly noisier. Regarding the entropy temperature α, PER variants require higher values. First, α decreases, then increases, before eventually converging.

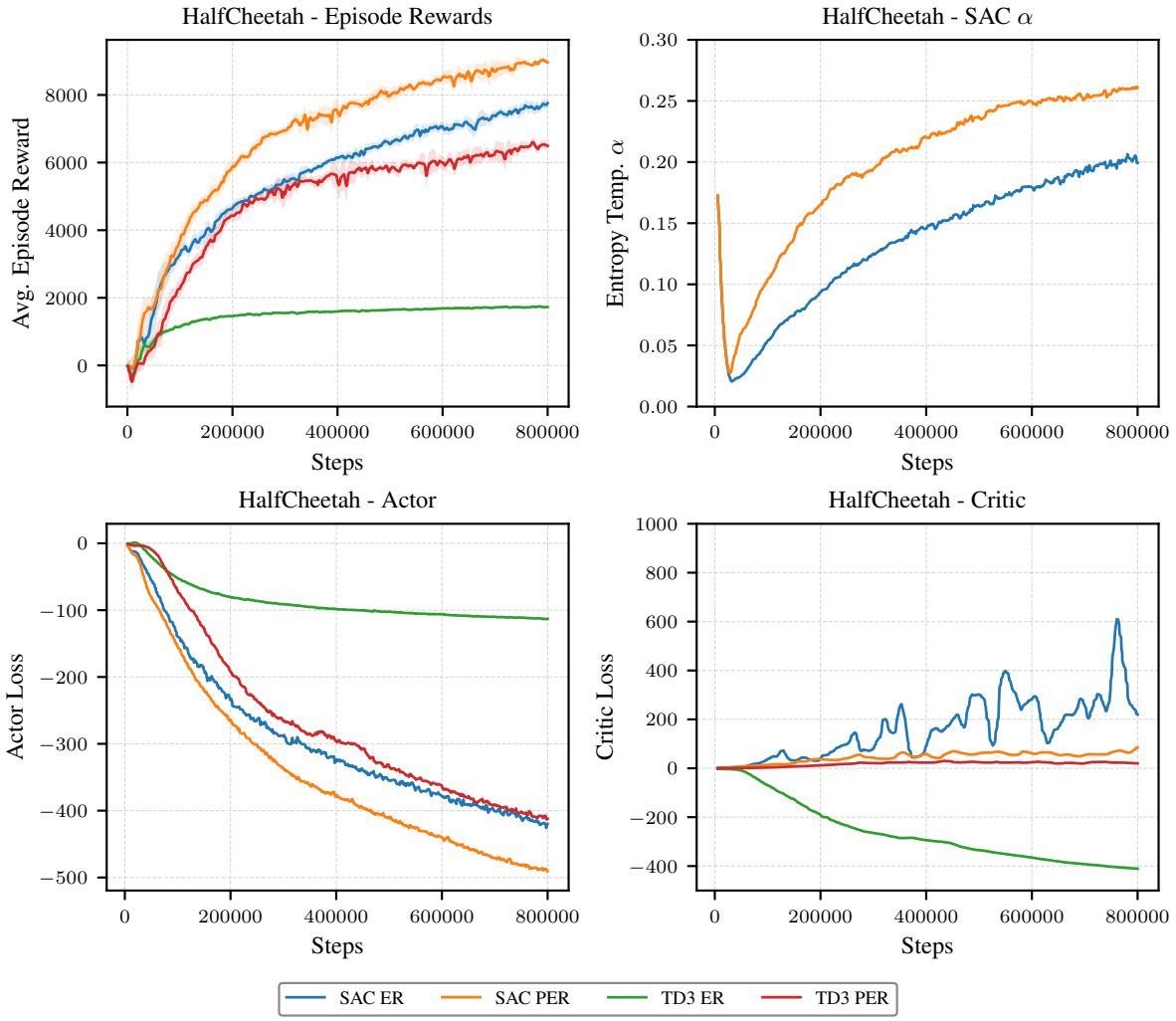

Figure 5: Training diagnostics for Half Cheetah environment. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

A.2 Hyperparameters

For reproducibility and completeness, the hyperparameters for all environments are listed below. SAC typically performs well without extensive hyperparameter tuning, whereas TD3 often requires more careful parameter selection, particularly when combined with PER. This is especially noticeable with respect to network architecture choices.

Table 3: Training hyperparameters for Pendulum

Parameter	**SAC**	**SAC**	**TD3**	**TD3**
Replay buffer type	ER	PER	ER	PER
Discount factor γ	0.98	0.98	0.98	0.98
Actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Critic learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Entropy coefficient α	0.2	0.2	–	–
Replay buffer size	10^5	10^5	10^5	10^5
Noise	–	–	$\sigma = 0.2$,clip=0.5	$\sigma = 0.2$,clip=0.5
Network architecture	$[256, 256]$	$[256, 256]$	$[128, 128]$	$[128, 128]$
Critic architecture	$[256, 256, 128]$	$[256, 256, 128]$	$[128, 128, 128]$	$[128, 128, 128]$

Table 4: Training hyperparameters for Lunar Lander

Parameter	**SAC**	**SAC**	**TD3**	**TD3**
Replay buffer type	ER	PER	ER	PER
Discount factor γ	0.99	0.99	0.99	0.99
Actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Critic learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Entropy coefficient α	0.2	0.2	–	–
Replay buffer size	2×10^5	2×10^5	2×10^5	2×10^5
Noise	–	–	$\sigma = 0.2$,clip=0.5	$\sigma = 0.2$,clip=0.5
Network architecture	$[256, 256]$	$[256, 256]$	$[256, 256]$	$[256, 256]$
Critic architecture	$[256, 256, 128]$	$[256, 256, 128]$	$[256, 256, 128]$	$[256, 256, 128]$

Table 5: Training hyperparameters for HalfCheetah

Parameter	**SAC**	**SAC**	**TD3**	**TD3**
Replay buffer type	ER	PER	ER	PER
Discount factor γ	0.99	0.99	0.99	0.99
Actor learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Critic learning rate	3×10^{-4}	3×10^{-4}	3×10^{-4}	3×10^{-4}
Entropy coefficient α	0.2	0.2	–	–
Replay buffer size	8×10^5	8×10^5	8×10^5	80^5
Noise	–	–	$\sigma = 0.2$,clip=0.5	$\sigma = 0.2$,clip=0.5
Network architecture	$[256, 256, 256]$	$[256, 256, 256]$	$[256, 256, 256]$	$[256, 256, 256]$
Critic architecture	$[256, 256, 256]$	$[256, 256, 256]$	$[256, 256, 256]$	$[256, 256, 256]$

B Hockey Environment Diagnostics

This appendix provides additional training diagnostics, hyperparameter settings, and tournament results for the hockey environment experiments described in Section 3.

B.1 Training Curves

Further training diagnostics are shown in Figure 6. The SAC entropy temperature α decays steadily for both variants, reflecting increasing policy confidence. The TD3 actor losses decrease monotonically into negative territory as the actor learns to maximize the critic's Q-values. The critic loss of TD3 Basic appears noisier than the other variants, which may be attributable to the uncorrelated white noise producing more inconsistent transition patterns in the replay buffer. Critic losses decrease across all variants towards the end of training, indicating stable convergence.

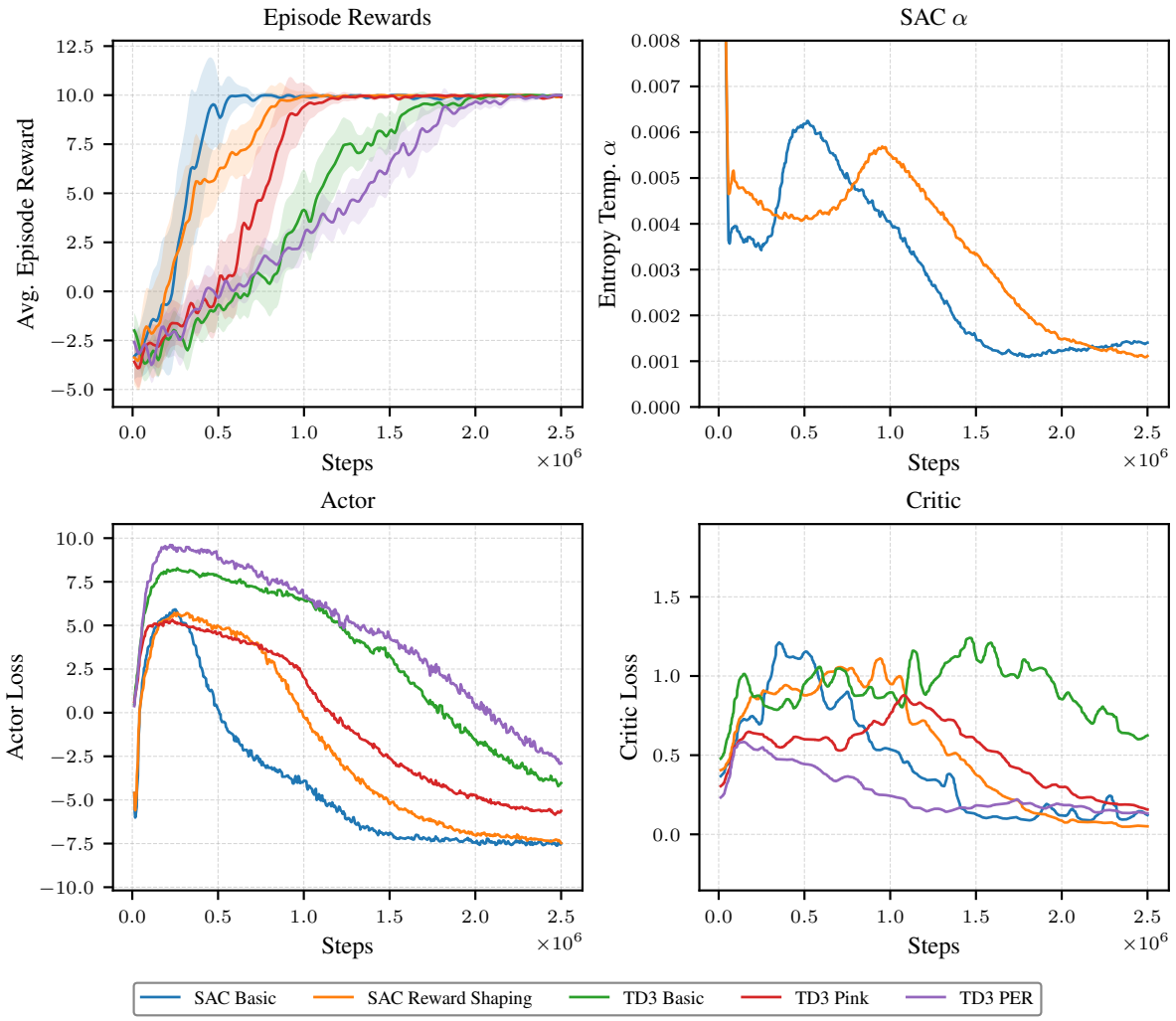

Figure 6: Training diagnostics for selected hockey environment configurations. Top left: average episode reward. Top right: SAC entropy temperature α. Bottom left: actor loss. Bottom right: critic loss.

B.2 Hyperparameters

Table 6: SAC hyperparameters.

Hyperparameter	**Value**
Discount factor γ	0.98
Soft update τ	0.005
Actor learning rate	3×10^{-4}
Critic learning rate	3×10^{-4}
α learning rate	1×10^{-4}
Initial entropy coefficient α	0.2
Target entropy	$-\lvert\mathcal{A}\rvert$
Actor architecture	$[512, 256]$
Critic architecture	$[512, 256, 128]$
Replay buffer size	10^6
Batch size	256
Learning starts	$10{,}000$
Reward shaping: touch puck	0.1
Reward shaping: puck direction	0.05

Table 7: TD3 hyperparameters.

Hyperparameter	**Value**
Discount factor γ	0.98
Soft update τ	0.005
Actor learning rate	3×10^{-4}
Critic learning rate	3×10^{-4}
Policy noise σ	0.2
Noise clip	0.5
Policy delay	2
Exploration noise (pink, $\beta = 1$)	0.15
Actor architecture	$[256, 256, 256]$
Critic architecture	$[256, 256, 128]$
Replay buffer size	10^6
Batch size	256
Learning starts	$10{,}000$
Reward shaping: touch puck	0.1
Reward shaping: puck direction	0.05

B.3 Tournament Results

The tournament was conducted as a full round-robin in which every participant played 1000 episodes against every other participant, once as the left and once as the right player. Rankings are determined by the TrueSkill lower confidence bound LCB $= \mu - 3\sigma$, where μ denotes the estimated skill and σ the uncertainty.

Both agents fine-tuned against the pre-trained opponent pool rank first and second by a clear margin, confirming the effectiveness of diverse opponent training. It is worth noting that all agents ranked below the fine-tuned variants were part of the pre-trained opponent pool during their training, meaning the top agents were explicitly optimized to beat them. Notably, SAC trained only against the weak and strong opponent still outperforms TD3 self-play, suggesting that TD3 self-play overfits to the strategies present in its own snapshot pool. All learned agents reliably surpass both basic opponents, which rank at the bottom of the leaderboard.

Table 8: Final tournament results ranked by TrueSkill LCB ($\mu - 3\sigma$). Each participant plays 1000 episodes against every other participant.

Rank	**Agent**	μ	σ	**LCB**	**W-L-D**	**Win Rate**
1	SAC (pre-trained pool)	41.72	3.34	31.72	6520-438-42	93.1%
2	TD3 (pre-trained pool)	36.59	3.04	27.48	5913-1044-43	84.5%
3	SAC (self-play)	32.51	2.80	24.09	4417-2440-143	63.1%
4	SAC (weak/strong)	29.12	2.91	20.39	3558-3251-191	50.8%
5	TD3 (self-play)	25.25	2.83	16.76	3251-3434-315	46.4%
6	TD3 (weak/strong)	21.76	2.89	13.10	2614-4232-154	37.3%
7	Basic Strong Opponent	18.34	3.37	8.22	405-6102-493	5.8%
8	Basic Weak Opponent	10.76	4.08	-1.49	339-6076-585	4.8%

List of Acronyms

DQN Deep Q-Network

PPO Proximal Policy Optimization

DDPG Deep Deterministic Policy Gradient

TD3 Twin Delayed Deep Deterministic Policy Gradient

SAC Soft Actor-Critic

ER Experience Replay

PER Prioritized Experience Replay

TD Temporal Difference

YAML YAML Ain't Markup Language